



Chapter 16

Least Squares Regression

By

Hyung Jae (Chris) Chang

Troy University



Introduction

- From a given data points (x, y) , we want to find an estimation function $\hat{y}(x)$.
- $\hat{y}(x)$ can be a function of several variables but assuming only one in this section
 - $\hat{y}(x)$: linear combination of basis functions, $f_i(x)$
 - α_i : parameters of the estimation function
 - Each basis function must be linearly independent

$$\hat{y}(x) = \sum_{i=1}^n \alpha_i f_i(x)$$

- Goal (least squares regression): Find parameters of the estimation function that minimize the total squared error

$$E = \sum_{i=1}^m (\hat{y} - y)^2 \text{ where } (\hat{y} - y) \text{ is individual errors or residuals denoted as } e_i$$

- If e is the vector containing all the individual errors, then we want to minimize $E = \|e\|_2^2$, which is L_2 norm.



Least Squares Regression Derivation (Linear Algebra)

- Estimation of data at each data point x_i :

$$\begin{aligned}\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ \hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\quad \dots \\ \hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).\end{aligned}$$

- $X \in \mathbb{R}^n$: a column vector such that the i th element of X contains the value of the i th x -data point, x_i .
- $\hat{Y}$: a column vector with elements, $\hat{Y}_i = \hat{y}(x_i)$
- β : a column vector such that $\beta_i = \alpha_i$
- $F_i(x)$: a function that returns a column vector of $f_i(x)$ computed on every element of x
- A : an $m \times n$ matrix such that the i th column of A is $F_i(x)$.
- Then $\hat{Y} = A\beta$



Least Squares Regression Derivation (Linear Algebra) (cont.)

- If Y is a column vector such that $Y_i = y_i$, the total squared error is given by $E = \|\hat{Y} - Y\|_2^2$.
- Verification
 - By substituting the definition of the L_2 norm which is the measure of distance.
 - Make E as small as possible $\rightarrow \hat{Y}$ and Y to be as close as possible
 - Note: in general, Y will not be in the range of A and there for $E > 0$.

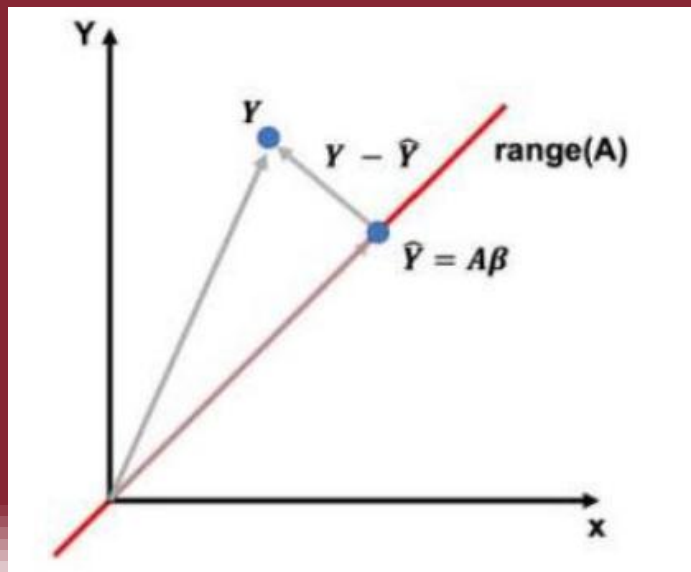


Illustration of the L_2 projection of Y on the range of A
* This is not a plot of the data points (x_i, y_i)



Least Squares Regression Derivation (Linear Algebra) (cont.)

- Observation: we want a vector $Y - \hat{Y}$ which is perpendicular to the vector $\hat{Y}$.
- Two vectors are perpendicular (or orthogonal) $\rightarrow$ when their dot-product is 0. ($\cos(\pi/2)$)
- Dot-product between two vectors, v and w , can be written as
 - $\text{dot}(v, w) = v^T w = |v| |w| \cos(\theta)$
 - $\hat{Y}$ and $\hat{Y} - Y$ are perpendicular $\rightarrow \text{dot}(\hat{Y}, Y - \hat{Y}) = 0$, therefore $\hat{Y}^T(Y - \hat{Y}) = 0$ which is equivalent to $(A\beta)^T(Y - A\beta) = 0$.
 - By distributive properties of vector multiplication, we can obtain $\beta^T A^T Y - \beta^T A^T A \beta = \beta^T (A^T Y - A^T A \beta) = 0$; $\beta=0$ for trivial solution so we use $A^T Y - A^T A \beta = 0$
- Solving for β gives the least squares regression formula:

$$\beta = (A^T A)^{-1} A^T Y$$



Least Squares Regression Derivation (Multivariate Calculus)

- Recall: the total error for m data points and n basis function

$$E = \sum_{i=1}^m e_i^2 = \sum_{i=1}^m (\hat{y}(x_i) - y_i)^2 = \sum_{i=1}^m \left(\sum_{j=1}^n \alpha_j f_j(x_i) - y_i \right)^2$$

which are n -dimensional paraboloid in α_k

- From calculus, the minimum of a paraboloid is where all the partial derivatives equal to zero.



Least Squares Regression in Python

- Recall the enumeration of the estimation of the data at each data point x_i .

$$\begin{aligned}\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ \hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\dots \\ \hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).\end{aligned}$$

- If the data were perfect, we get the following system of equations:

$$\begin{aligned}y_1 &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ y_2 &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\dots \\ y_m &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).\end{aligned}$$

- A is defined as previously $\rightarrow Y = A\beta$.
- Since the data is not perfect, we use the least square regression to obtain a solution, $\beta = (A^T A)^{-1} A^T Y$.



Example

TRY IT! Consider the artificial data created by $x = \text{np.linspace}(0, 1, 101)$ and $y = 1 + x + x * \text{np.random.random}(\text{len}(x))$. Do a least squares regression with an estimation function defined by $\hat{y} = \alpha_1 x + \alpha_2$. Plot the data points along with the least squares regression. Note that we expect $\alpha_1 = 1.5$ and $\alpha_2 = 1.0$ based on this data. Due to the random noise we added into the data, your results may be slightly different. In the next few subsections, we will see how we solve this problem using different approaches.



Least Squares Regression in Python (Using Direct Inverse Method)

```
In [1]: import numpy as np
        from scipy import optimize
        import matplotlib.pyplot as plt

        plt.style.use("seaborn-poster")

In [2]: # generate x and y
        x = np.linspace(0, 1, 101)
        y = 1 + x + x * np.random.random(len(x))

In [3]: # assemble matrix A
        A = np.vstack([x, np.ones(len(x))]).T

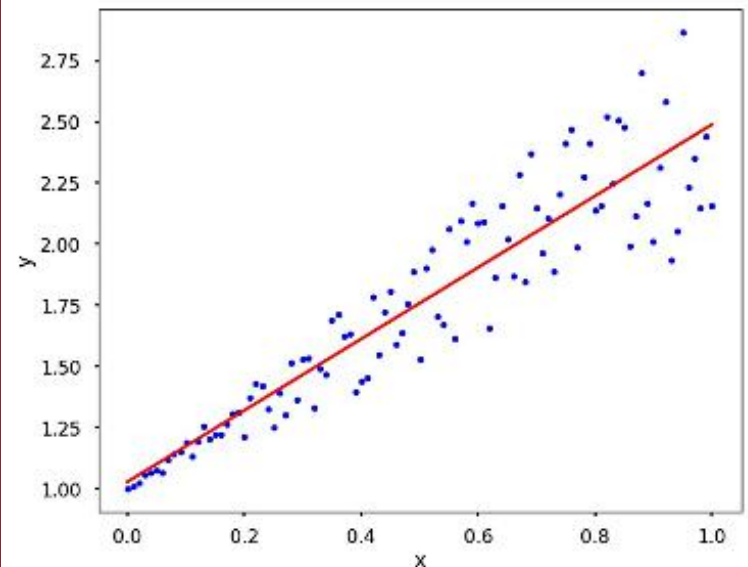
        # turn y into a column vector
        y = y[:, np.newaxis]

In [4]: # Direct least squares regression
        alpha = np.dot((np.dot(np.linalg.inv(np.dot(A.T, A)), A.T)), y)
        print(alpha)
```

$$\beta = (A^T A)^{-1} A^T Y$$

```
[[1.459573 ]
 [1.02952189]]
```

```
In [5]: # plot the results
        plt.figure(figsize = (10,8))
        plt.plot(x, y, "b.")
        plt.plot(x, alpha[0]*x + alpha[1], "r")
        plt.xlabel("x")
        plt.ylabel("y")
        plt.show()
```





Least Squares Regression in Python (Using `numpy.linalg.lstsq`)

- NumPy has already implemented the least squares methods.

```
In [7]: alpha = np.linalg.lstsq(A, y, rcond=None)[0]  
        print(alpha)
```

```
[[1.459573  ]  
 [1.02952189]]
```



Least Squares Regression in Python (Using Optimize.curve_fit)

- From SciPy library, we can use `optimize.curve_fit` function.
- We do not need to turn `y` into a column vector.

```
In [8]: # generate x and y
        x = np.linspace(0, 1, 101)
        y = 1 + x + x * np.random.random(len(x))

In [9]: def func(x, a, b):
        y = a*x + b
        return y

        alpha=optimize.curve_fit(func, xdata=x, ydata=y)[0]
        print(alpha)

[1.44331612  1.0396133 ]
```



Least Squares Regression for Nonlinear Functions

- A least squares regression requires that the estimation function be a linear combination of basis functions.
- There are some functions that can't be put in this form, but where a least squares regression is still appropriate.
- How?
 - By taking advantages of the properties of logarithms and transform the nonlinear function into a linear function.
 - By using `curve_fit` function from Scipy to estimate directly the parameters for the nonlinear function using least square.



Using Log Tricks for Exponential Functions

- Assume that we have a function $\hat{y}(x) = \alpha e^{\beta x}$ and data x and y
→ perform the Least Squares Regression to find α and β .
- Take log on both sides → $\log(\hat{y}(x)) = \log \alpha + \beta x$
- If $\tilde{y}(x) = \log(\hat{y}(x))$ and $\tilde{\alpha} = \log \alpha$, $\tilde{y}(x) = \tilde{\alpha} + \beta x$ which is a linear form for which we can perform a least squares regression to find $\tilde{y}(x)$, $\tilde{\alpha}$ and β .
- Then, we can recover α by using $\alpha = e^{\tilde{\alpha}}$.



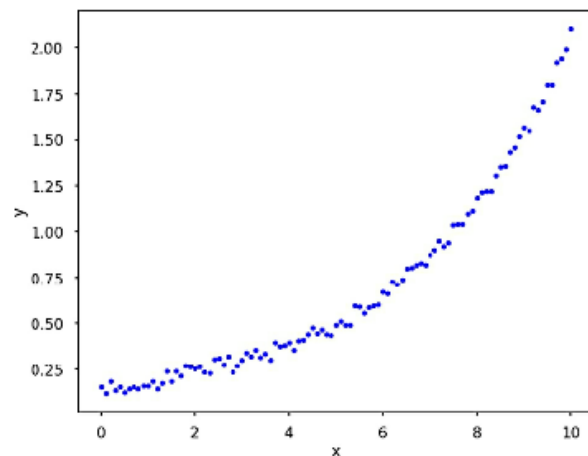
Using Log Tricks for Exponential Functions

For the example below, we will generate data using $\alpha=0.1$ and $\beta=0.3$.

```
In [1]: import numpy as np
        from scipy import optimize
        import matplotlib.pyplot as plt
        plt.style.use("seaborn-noraster")
```

```
In [2]: # let's generate x and y, and add some noise into y
        x = np.linspace(0, 10, 101)
        y = 0.1*np.exp(0.3*x) + 0.1*np.random.random(len(x))
```

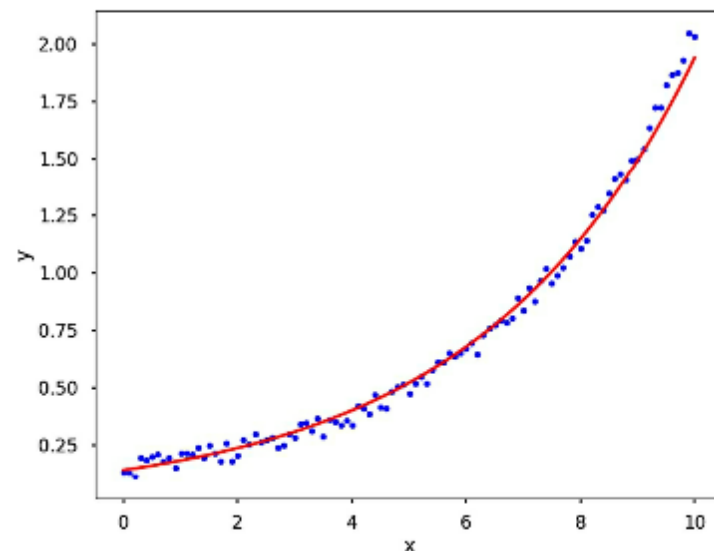
```
In [3]: # Let's have a look of the data
        plt.figure(figsize = (10,8))
        plt.plot(x, y, "b.")
        plt.xlabel("x")
        plt.ylabel("y")
        plt.show()
```



```
In [4]: A = np.vstack([x, np.ones(len(x))]).T
        beta, log_alpha = np.linalg.lstsq(A, np.log(y), rcond = None)[0]
        alpha = np.exp(log_alpha)
        print(f"alpha={alpha}, beta={beta}")
```

alpha=0.13973103064296616, beta=0.26307478591152406

```
In [5]: # Let's have a look of the data
        plt.figure(figsize = (10,8))
        plt.plot(x, y, "b.")
        plt.plot(x, alpha*np.exp(beta*x), "r")
        plt.xlabel("x")
        plt.ylabel("y")
        plt.show()
```





Using Log Tricks for Power Functions

- Function $\hat{y}(x) = bx^m$ and data for x and y .
- By taking log on both side, we get $\log(\hat{y}(x)) = m \log x + \log b$ which is a linear form.
- And we solve this as a linear regression.



Using Polynomial Regression

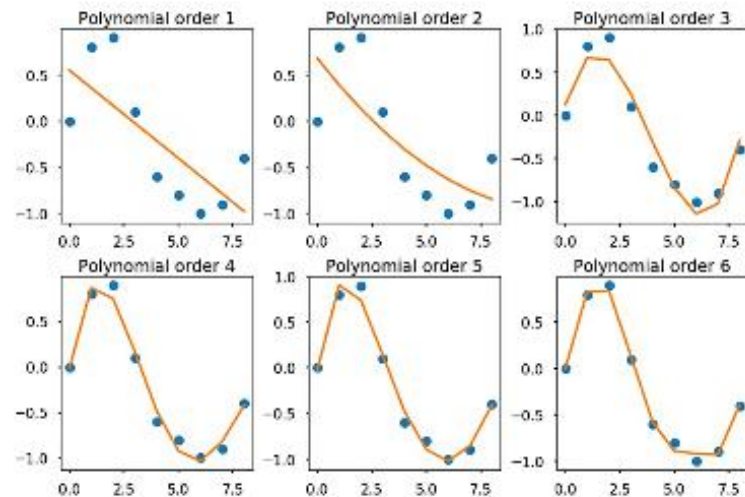
- Use polynomial and least squares to fit a nonlinear function.
- Polynomials:
$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_2 x^2 + a_1 x^1 + a_0$$
where $a_n, a_{n-1}, \dots, a_2, a_1$ are the real number coefficients and n (a non-negative integer) is the order or degree of the polynomial.
- In Python, `numpy.polyfit` is used to obtain the coefficients of different order polynomials with the least squares.
- With the coefficients, `numpy.polyval` is used to obtain the specific values



Using Polynomial Regression (cont.)

```
In [6]: x_d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8])
        y_d = np.array([0, 0.8, 0.9, 0.1, -0.6, 0.8, -1, -0.9, -0.4])

        plt.figure(figsize = (12, 8))
        for i in range(1, 7):
            # get the polynomial coefficients
            y_est = np.polyfit(x_d, y_d, i)
            plt.subplot(2,3,i)
            plt.plot(x_d, y_d, "o")
            # evaluate the values for a polynomial
            plt.plot(x_d, np.polyval(y_est, x_d))
            plt.title(f"Polynomial order {i}")
        plt.tight_layout()
        plt.show()
```





Using OPTIMIZE.CURVE_FIT from SCIPY

```
In [7]: # let's define the function form
def func(x, a, b):
    y = a*np.exp(b*x)
    return y
alpha, beta = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(f"alpha={alpha}, beta={beta}")
```

alpha=0.12663549356730994, beta=0.27760076897453045

```
In [8]: # Let's have a look of the data
plt.figure(figsize = (10,8))
plt.plot(x, y, "b.")
plt.plot(x, alpha*np.exp(beta*x), "r")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

